开源软件基础

Python Requests

OSCAR Team, School of Software, Dalian University of Technology

November 28, 2017

HTTP 基础知识

HTTP

- GET 向指定的资源发出“显示”请求。使用 GET 方法应该只用在读取数据，而不应当被用于产生“副作用”的操作中，例如在 Web Application 中。其中一个原因是 GET 可能会被网络蜘蛛等随意访问。
- HEAD 与 GET 方法一样，都是向服务器发出指定资源的请求。只不过服务器将不传回资源的本文部分。它的好处在于，使用这个方法可以在不必传输全部内容的情况下，就可以获取其中“关于该资源的信息”(元信息或称元数据)。

HTTP 基础知识

HTTP

- POST 向指定资源提交数据，请求服务器进行处理（例如提交表单或者上传文件）。数据被包含在请求本文中。这个请求可能会创建新的资源或修改现有资源，或二者皆有。
- PUT 向指定资源位置上传其最新内容。
- DELETE 请求服务器删除 Request-URI 所标识的资源。
- TRACE 回显服务器收到的请求，主要用于测试或诊断。

HTTP 基础知识

HTTP

- OPTIONS 这个方法可使服务器传回该资源所支持的所有 HTTP 请求方法。用'*' 来代替资源名称，向 Web 服务器发送 OPTIONS 请求，可以测试服务器功能是否正常运作。
- CONNECT HTTP/1.1 协议中预留给能够将连接改为管道方式的代理服务器。通常用于 SSL 加密服务器的链接（经由非加密的 HTTP 代理服务器）。

数据准备

```
$head -n 10 robots.txt # from taobao.com/robots.txt

User-agent:  Baiduspider
Allow:  /article
Allow:  /oshtml
Allow:  /wenzhang
Disallow:  /product/
Disallow:  /

User-Agent:  Googlebot
Allow:  /article
Allow:  /oshtml
```

数据准备

数据准备

```
$cat index.html # use type in win32
<html>
    <head>
    </head>
    <body>
        <h1>Title</h1>
        <p> This is a paragraph</p>
        <table>
            <tr>
                <td>A11</td> <td>A12</td> <td>A13</td>
            </tr>
            <tr>
                <td>A21</td> <td>A22</td> <td>A23</td>
            </tr>
            <tr>
                <td>A31</td> <td>A32</td> <td>A33</td>
            </tr>
        </table>
    </body>
</html>

$python3 -m http.server
```

本地 vs. 远程

我发起疯来，连自己都爬

Outline

机器人协议

robots.txt（统一小写）是一种存放于网站根目录下的 ASCII 编码的文本文件，它通常告诉网络搜索引擎的漫游器（又称网络蜘蛛），此网站中的哪些内容是不应被搜索引擎的漫游器获取的，哪些是可以被漫游器获取的。因为一些系统中的 URL 是大小写敏感的，所以 robots.txt 的文件名应统一为小写。robots.txt 应放置于网站的根目录下。如果想单独定义搜索引擎的漫游器访问子目录时的行为，那么可以将自定的设置合并到根目录下的 robots.txt，或者使用 robots 元数据（Metadata，又称元数据）。[1]

[1] https://zh.wikipedia.org/wiki/Robots.txt

机器人协议

协议特点

- robots.txt 协议并不是一个规范，而只是约定俗成的，所以并不能保证网站的隐私。
- 用字符串比较来确定是否获取 URL，所以目录末尾有与没有斜杠“/”表示的是不同的 URL。
- 允许使用类似"Disallow: *.gif" 这样的通配符。

机器人协议

允许所有的机器人：

```
User-agent: *
Disallow:
```

另一写法

```
User-agent: *
Allow:/
```

机器人协议

仅允许特定的机器人：

```
User-agent: name_spider
Allow:
```

拦截所有的机器人

```
User-agent: *
Disallow: /
```

机器人协议

禁止所有机器人访问特定目录

```
User-agent: *
Disallow: /cgi-bin/
Disallow: /images/
Disallow: /tmp/
Disallow: /private/
```

仅禁止坏爬虫访问特定目录

```
User-agent: BadBot
Disallow: /private/
```

机器人协议

禁止所有机器人访问特定目录

```
User-agent: *
Disallow: /*.php$
Disallow: /*.js$
Disallow: /*.inc$
Disallow: /*.css$
```

如何遵守协议

- 人工阅读 robots.txt
- 使用 urllib.robotparser

解析 robots.txt

```
>>> import urllib.robotparser
>>> rp = urllib.robotparser.RobotFileParser()
>>> rp.set_url("http://127.0.0.1:8000/robots.txt")
>>> rp.read()
>>> rp.can_fetch("Baiduspider", "http://127.0.0.1:8000/")
False
>>> rp.can_fetch("Baiduspider", "http://127.0.0.1:8000/article/")
True
```

Outline

Apache2 License

A large number of open source projects you find today are GPL Licensed. While the GPL has its time and place, it should most certainly not be your go-to license for your next open source project.

Apache2 License

A project that is released as GPL cannot be used in any commercial product without the product itself also being offered as open source.

Apache2 License

The MIT, BSD, ISC, and Apache2 licenses are great alternatives to the GPL that allow your open-source software to be used freely in proprietary, closed-source software.
Requests is released under terms of Apache2 License.

Outline

Requests: HTTP for Humans

```
>>> r = requests.get('https://api.github.com/user', auth=('user', 'pass'))
>>> r.status_code
200
>>> r.headers['content-type']
'application/json; charset=utf8'
>>> r.encoding
'utf-8'
>>> r.text
u'{"type":"User"...'
>>> r.json()
{u'private_gists': 419, u'total_private_repos': 77, ...}
```

User Testimonials

Twitter, Spotify, Microsoft, Amazon, Lyft, BuzzFeed, Reddit, The NSA, Her Majesty's Government, Google, Twilio, Runscope, Mozilla, Heroku, PayPal, NPR, Obama for America, Transifex, Native Instruments, The Washington Post, SoundCloud, Kippt, Sony, and Federal U.S. Institutions that prefer to be unnamed claim to use Requests internally.

User Testimonials

Requests is one of the most downloaded Python packages of all time, pulling in over 13,000,000 downloads every month. All the cool kids are doing it!
Requests is ready for today's web.
Requests officially supports Python 2.62.7 & 3.43.7, and runs great on PyPy.

Outline

发起连接

```
import requests
r = requests.get("http://127.0.0.1:8000")
print(r.text)
print(r.encoding)
print(r.url)
print(r.status_code)
```

GET 参数

```
import requests
import json
payload = {'key1': 'value1', 'key2': 'value2'}
r = requests.get('http://httpbin.org/get', params=payload)
result = json.loads(r.text)
print(result['args'])
```

编码问题

```
r = requests.get("http://127.0.0.1:8000")
r.encoding = "UTF-8"
print(r.text)
```

POST 参数

```
>>> r = requests.post('http://httpbin.org/post', data = {'key':'value'})
```

Outline

正规式

正规式

正则表达式，又称正规表示式、正规表示法、正规表达式、规则表达式、常规表示法（英语：Regular Expression，在代码中常简写为 regex、regexp 或 RE），是计算机科学的一个概念。正则表达式使用单个字符串来描述、匹配一系列匹配某个句法规则的字符串。在很多文本编辑器里，正则表达式通常被用来检索、替换那些匹配某个模式的文本。[a]

[a] https://zh.wikipedia.org/zh-cn/

正规式

```
>>> import re
>>> re.findall(r'\bf[a-z]*', 'which foot or hand fell fastest')
['foot', 'fell', 'fastest']
>>> re.sub(r'(\b[a-z]+) \1', r'\1', 'cat in the the hat')
'cat in the hat'
```

正规式

```
>>> html = '''\
<html>
    <head>
    </head>
    <body>
        <h1>Title</h1>
        <p> This is a paragraph</p>
        <table>
            <tr>
                <td> <a href='a.html'>link a</a> </td>
            </tr>
        </table>
        <a href='b.html'>link b</a>
    </body>
</html>
'''
>>> re.findall("href='(.*?)'", html)
['a.html', 'b.html']
```

匹配字符

字符模式

- [a-zA-Z] 匹配某个字符
- ^和 $ 匹配首尾
- \^和 \$ 字面值
- . 匹配任意字符，\. 匹配点号
- \b 匹配单词边界
- \w 匹配非空字符
- \s 匹配空格

匹配重复

匹配重复

- * 表示任意次数，* 表示字面值
- + 表示 1 次或多次，\+ 表示字面值
- *? 和 +? 表示最短匹配
- {3} 和 {4,6} 表示重复 3 次和 4–6 次

匹配重复

```
>>> import re
>>> re.match(r'ab*', 'abbbb').group()
'abbbb'

>>> re.match(r'ab*?', 'abbbb').group()
'a'

>>> re.match(r'a\d{3,4}', 'a1234567').group()
'a1234'
```

模式选择

```
>>> #使用|表示或，\|表示字面值
>>> import re
>>> re.match(r'(abc)|(def)', 'abc').group()
'abc'
```

match 与 findall

match vs. findall

- match 从首字母开始匹配
- 还有个 search，与 match 类似，从任意处开始匹配
- findall 匹配所有子串

match 与 findall

```
>>> import re
>>> p = re.compile('\d+')
>>> p.findall('12 drummers drumming, 11 pipers piping,
['12', '11', '10']

>>> p.match('12 drummers drumming, 11 pipers piping, 1
'12'
```

分组捕获

Grouping

- () 表示分组
- \ 数字进行引用

分组捕获

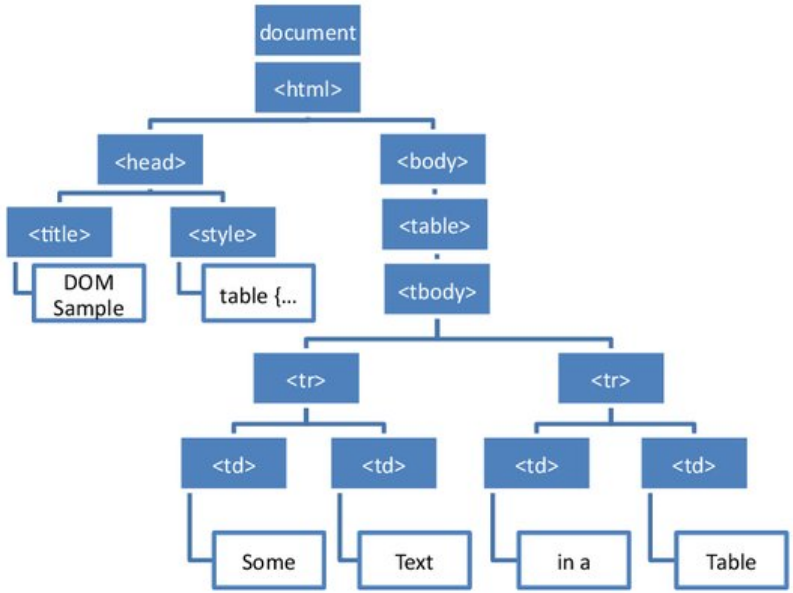
```
>>> re.match(r'x(\d+) = \1', 'x123 = 123').group()
'x123 = 123'
>>> re.match(r'x(\d+) = \1', 'x123 = 123').groups()
('123',)
>>> p = re.compile('(a(b)c)d')
>>> m = p.match('abcd')
>>> m.group(0)
'abcd'
>>> m.group(1)
'abc'
>>> m.group(2)
'b'
>>> re.findall(r'(.)\1', '明明亮亮蛋蛋')
['明', '亮', '蛋']
```

正规式的局限

- 无法描述复杂结构
- 回顾编译原理，正规式将文本流转化为记号流
- 还需要语法分析和语义分析构造 AST
- HTML 中仅用 href 过滤网址可能无法知道在网页中的位置
- 需要构造 DOM 树

DOM 树


```
<!DOCTYPE html>
<html>
    <head>
        <title>DOM Sample</title>
        <style type="text/css">
            table {
                border: 1px solid black;
            }
        </style>
    </head>
    <body>
        <table>
            <tbody>
                <tr>
                    <td>Some</td>
                    <td>Text</td>
                </tr>
                <tr>
                    <td>in a</td>
                    <td>Table</td>
                </tr>
            </tbody>
        </table>
    </body>
</html>
```



抽象语法树

3 + 1 * 2 * 4 + 5 ⟺